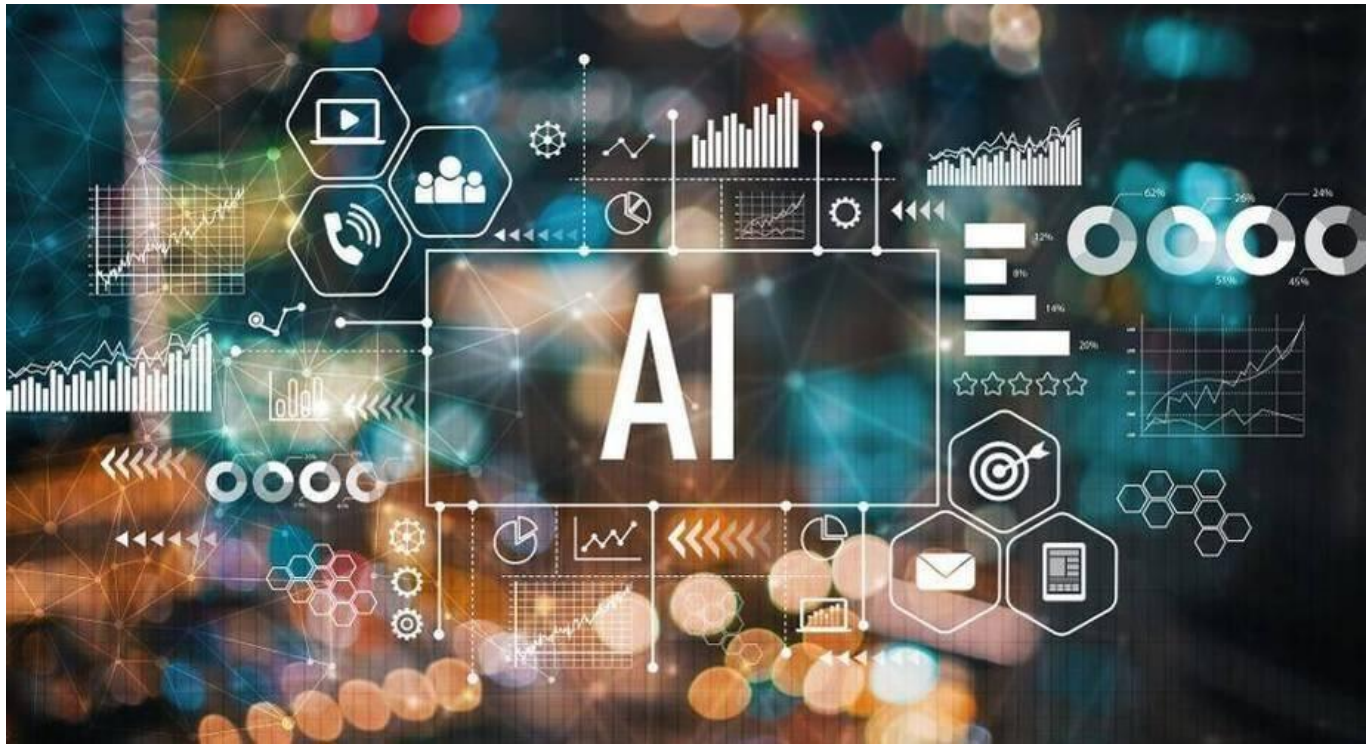
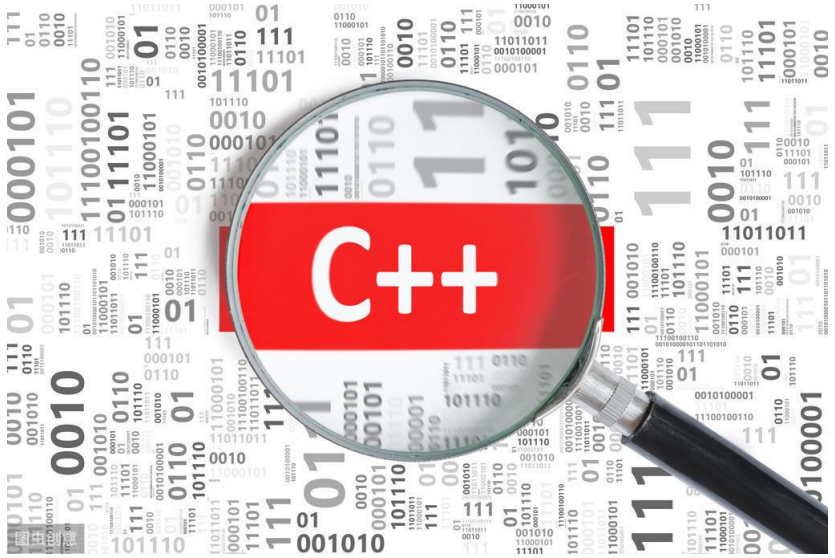

Haskell
JavaScript
C++
Perl
Ruby
Java
Python
Objective-C
Pascal
Assembly
Scratch
Prolog
Lua
TCL
Visual Basic
Mathematica
ColdFusion
FoxPro
Delphi
MATLAB
Forth
PL/SQL
Swift
Ada
Logo
Fortran
Lisp
Groovy
PHP
Scheme
COBOL
ABAP
Erlang
Yacc
Clarion
Transact-SQL
PostScript
OpenEdge ABL
Max/MSP
ML
F#
Tcl
Io
Go
Dart
Scala
SAS
C#
PL/I
cT

C++语言程序设计


AI


C++

第12章 异常处理

✧学习目标

1. 掌握try-throw-catch异常处理机制；
2. 能使用异常处理机制处理程序中常见的异常；
3. 了解标准异常处理类的内容；
4. 能用标准异常处理类处理new、越界类型的异常；



12.1 什么是异常?

- 无法打开某个事先定义的文件
- 网络连接可能突然中断
- 数据库断开
- 用户可能会意外输入一个不正确的值
- ……









12.1 异常的概念

西南交通大学
Southwest Jiaotong University

- **异常（exception）：程序运行时的错误**

  注意区别：*语法错误、逻辑错误、运行异常*

- **异常处理：对这些运行时发生的错误进行适当地处理**

- **异常处理的作用：在一定程度上可以保证程序的健壮性。**



12.1 异常的产生的原因?

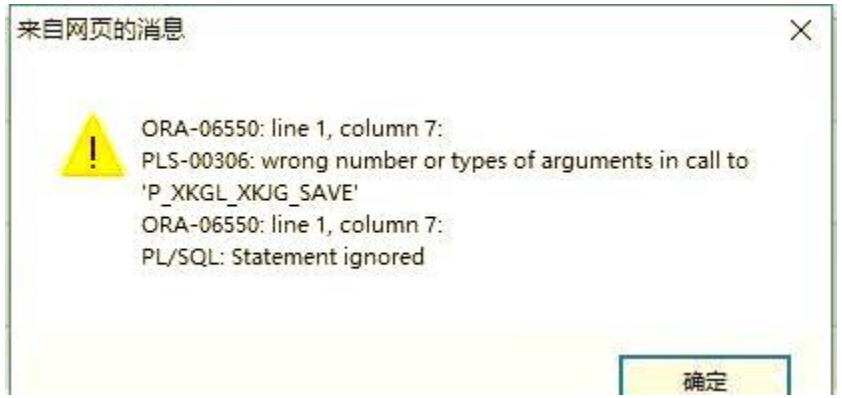
➢ 程序运行时的错误

程序编写本身的BUG

```
FILE *fp = fopen("d:\\tt.dat","rb");
int *p;
fread(p,10,10,fp);
```

➢ 不可控的错误

```
int ExeSelect(String sSQL,TADOQuery *pQue)
{
    pQue->SQL->Add(sSQL);
    pQue->Open();  // 访问数据库
 }
```

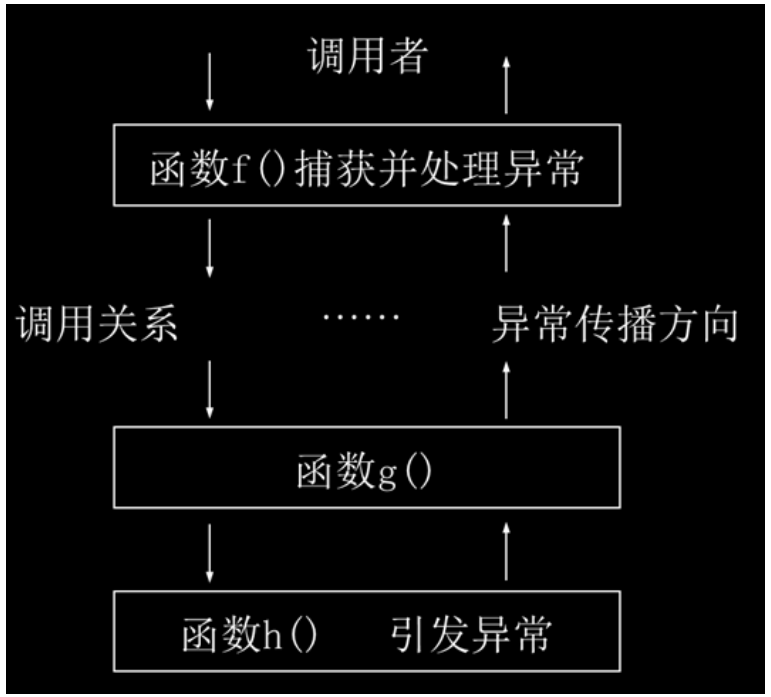


12.1 异常的概念

西南交通大学 Southwest Jiaotong University

在C++中，发生异常时，错误信息传递流程：

- 抛出一个错误信息，把它传递给上一级的调用函数来解决
- 上一级函数解决不了，再传给其上一级函数，由其上一级函数处理
- 如此逐级上传，直到最高一级函数
- 如果无法处理，系统会自动调用terminate，由它调用abort终止程序。



exit() 在中止程序运行前，会关闭被程序打开的文件、调用全局和static类型对象的析构函数等，abort()什么都不做。

12.1 异常的概念

西南交通大学 Southwest Jiaotong University

C++ 为异常的检测和处理提供 3 种途径：

① 让“用户”知道程序出现了异常，允许“用户”选择异常处理方式，并继续使用程序。

② 让“用户”知道程序出现了异常，退出程序的执行，并做好系统资源回收等“善后”工作，尽量不影响计算机系统中其它程序的正常运行。

③ 在程序发生异常时，能够根据预先设定的异常处理策略进行异常处理，能够在不打扰“用户”的情况下继续程序的运行。

西南交通大学 Southwest Jiaotong University

大家以往如何预见异常和判断异常?

以往的异常捕获方式是利用 if 语句

```
float quotient(int a, int b) { return a/(float)b; }
  …
cin>>a>>b;
if (b==0) //捕获异常
    cout<<"Divide 0 !"<<endl;
else
    cout<<a<<"/"<<b<<"="<<quotient(a,b);
```

12.1 异常的概念

西南交通大学 Southwest Jiaotong University

这种处理机制缺点?

① 每使用quotient()一次, 就必须利用 if 语句检查一次，使得程序对正常执行过程的描述与对异常的处理交织在一起，程序的易读性不好。

② 若异常信息在函数中返回，会破坏程序的逻辑性。

- *原来没有返回值的函数，要定义成返回值*
- *对原来有返回值的函数无法定义异常信息返回*
- *构造函数? 析构函数?*



★12.2 C++异常处理机制

抛掷异常的程序段

……

throw　表达式;

……

捕获并处理异常的程序段

try

　复合语句

catch（异常类型声明）

　复合语句

catch（异常类型声明）

　复合语句

　…

★12.2 C++异常处理机制

- 异常处理机制的主体有两大部分：异常抛出区try，异常处理区catch，二者作为一个整体出现，构成try-catch结构。

```
try
{
    //程序执行语句序列;
    throw     (异常类型表达式);
}
```

异常抛出区

```
catch (异常类型 1)
      {//异常处理 }
catch (异常类型 2)
      {//异常处理 }
   …
catch (…)       //前面没有列举的异常类型
      {//异常处理 }
```

异常处理区

```
//try-throw-catch 后继续执行的语句
```



阅读P386

```
/*************************************************************
* 程序名：p12_1.cpp                                          *
* 功能：  带异常处理的求两个数的商                           *
*************************************************************/
# include <iostream>
using namespace std;
float quotient(int a,int b) throw(char *) {
  if (b==0)       //捕获异常
      throw("Divide 0 !");
  else
      return a/(float)b;
}
int main( ) {
    int a,b;
    cout<<"Input a, b: ";
    cin>>a>>b;
    try
    {       cout<<a<<"/"<<b<<"="<<quotient(a,b);   }
     catch(char *ErrS)
    {       cerr<<ErrS<<endl;          }
    return 0;
}
```

运行结果:

```
Input a, b : 1  3↙
1/3=0.333333
Input a, b, : 3  0↙
Divide 0 !
```



```
float quotient(int a,int b) throw(char *) {
   if (b==0)       //捕获异常
        throw("Divide 0 !");
   else
        return a/(float)b;
}
int main( ) {
     int a,b;
     cout<<"Input a, b: ";
     cin>>a>>b;
     try
     {       cout<<a<<"/"<<b<<"="<<quotient(a,b);   }
      catch(char *ErrS)
     {       cerr<<ErrS<<endl;          }
     return 0;
}
```

思考?

- Throw什么?
- 类型有关系么?
- Catch 如何匹配?
- ……



★12.2 C++异常处理机制

- 异常类型可以是基本数据类型、构造数据类型
- 还可以是类类型
- 异常类型为…表示可以捕获任意类型的异常

但catch(…)不能出现在其他捕获语句之前，否则出现编译错误。

```
try {
        statements;
}catch (Exception1 exVar1) {
        handler for exception1;
}catch (Exception2 exVar2) {
        handler for exception2;
}catch (…) {
        handler for exceptionN;
}
```

★12.2 C++异常处理机制

异常处理的执行过程如下：

(1) 执行try块中的程序语句序列；

(2) 如果执行期间没有执行到throw()(没有引起异常)，跳过异常处理区的catch语句块，程序向下执行；

(3) 若执行期间引起异常，执行throw()语句抛出异常，进入异常处理区，将throw() 抛出的异常类型表达式(对象)依次与catch()中的类型匹配，获得匹配的catch子句将捕获并处理异常。继续执行异常处理区后的语句；

(4) 如果未找到匹配(异常未捕获到)，自动调用结束函数terminate()，其缺省功能是调用abort()终止程序。



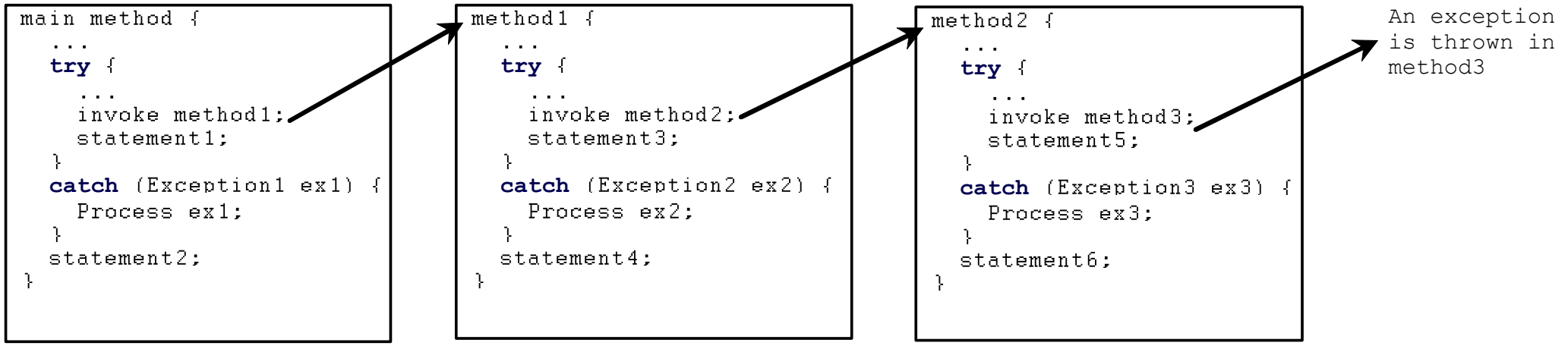
★12.3 异常处理嵌套与重抛异常

通过函数嵌套调用形成了**异常处理嵌套**。

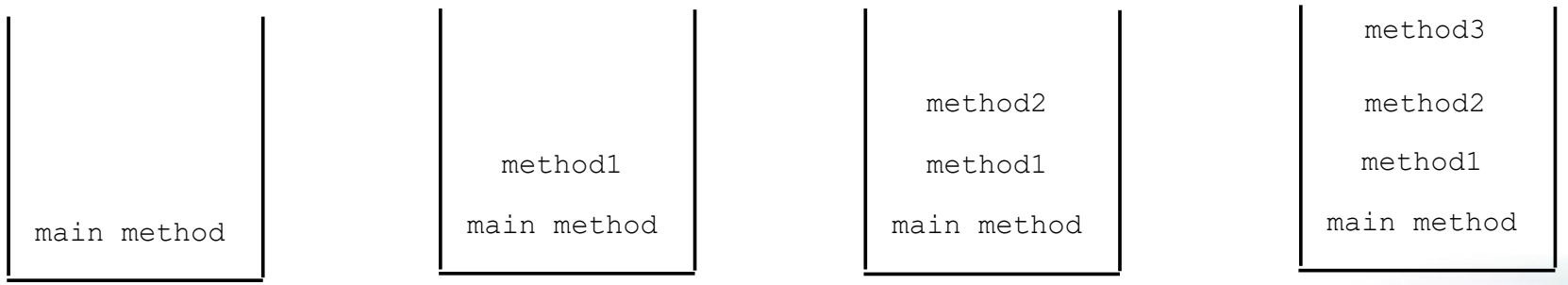
在这种情况下，最底层函数所抛出的异常首先在内层catch语句序列中依次查找与之匹配的处理，如果内层不能捕获，则内层函数抛出的异常**逐层向外**传递，最后回到主程序中。



12.2 C++异常处理机制



Call Stack



阅读P387

```
1   /*************************************************************
2   * 程序名：p12_2.cpp                                          *
3   * 功能：  带异常处理嵌套的求二元一次方程的解                  *
4   *************************************************************/
5   # include <cmath>
6   # include <iostream>
7   # include <stdexcept>
8   using namespace std;
9   struct Res {
10      float x1,x2;
11  };
12  Res resolution(int a,int b,int c) throw(int);
13  int main() {
14      int a,b,c;
15      Res r;
16      cout<<"Input a, b,c :";
17      cin>>a>>b>>c;
18      try  //异常侦测区
19      {
20          r=resolution(a,b,c);
21              cout<<"x1="<<r.x1<<"\tx2="<<r.x2<<endl;
22      }
```

```
22  catch (int)
23      {
24              cerr<<"Sqrt Negative Exception"<<endl;
25      }
26      catch(...)
27      {
28          cerr<<"unexpected or rethrow exception!"<<endl;
29      }
30     return 0;
31  }
32  float quotient(int a,int b) throw(char *)
33  {
34    if (b==0)       //捕获异常
35          throw("Divide 0 !");
36    else
37          return a/(float)b;
38  }
39  Res resolution(int a,int b,int c) throw(int)
40  {
41    Res tmpr;
```

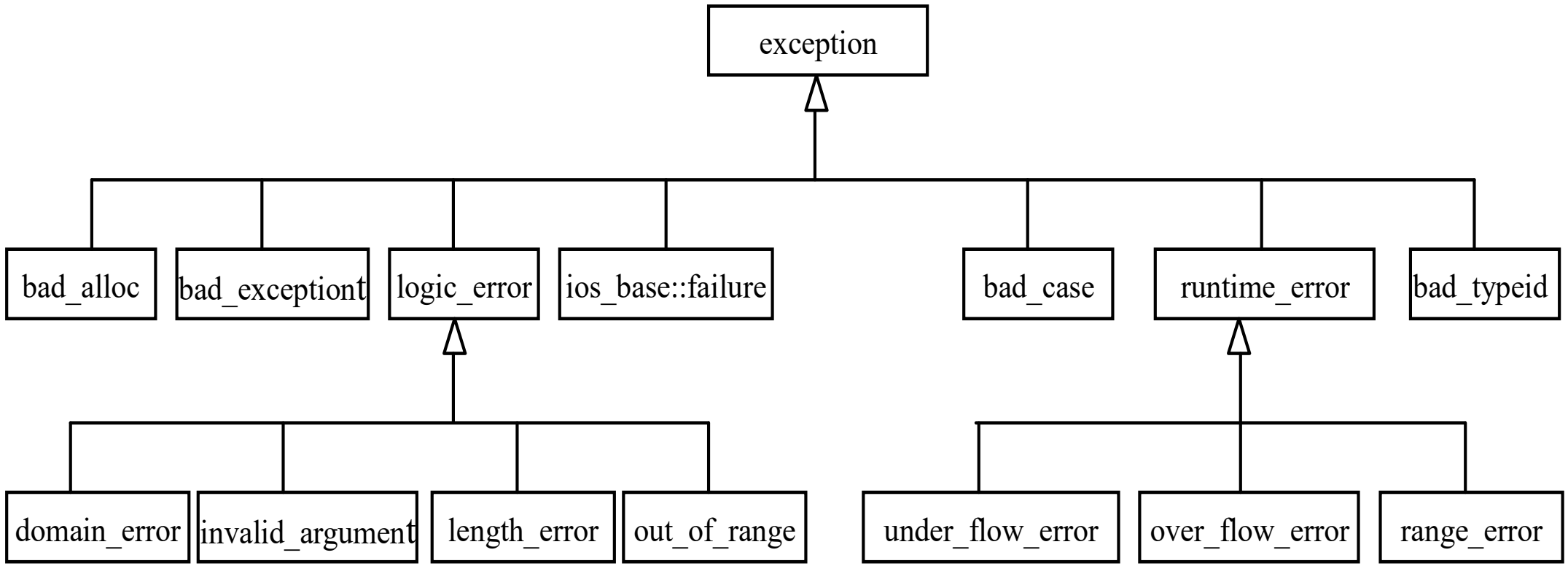
```
42  try
43    {
44      if(a==0&&b!=0)
45        {
46                tmpr.x1=tmpr.x2=quotient(c,b);
47                return tmpr;
48        }
49      if(b*b-4*a*c<0)
50           throw (b);
51      tmpr.x1=quotient(-b+sqrt(b*b-4.0*a*c),2*a);
52      tmpr.x2=quotient(-b-sqrt(b*b-4.0*a*c),2*a);
53      return tmpr;
54    }
55    catch(char *ErrS)
56    {
57          cerr<<ErrS<<endl;
58          //exit(0);
59          //throw;
60    }
61  }
```

运行结果:

```
Input a, b,c : 1  2  3↙
Sqrt Negative Exception!
Input a, b,c : 0  0  3↙
Divide 0 !
x1=5.88749e-039, x2=1.8347e-039
Input a, b,c : 1 -3 2↙
x1=2    x2=1
```

★12.4 标准异常处理类

C++提供了标准异常处理类库，它用来抛出C++标准库中函数执行时的异常。C++标准异常处理类的层次结构图如下图所示：



★C++标准库各种异常类所代表的异常

异常类	头文件	异常的含义
bad_alloc	exception	用new动态分配空间失败
bad_cast	new	执行dynamic_cast失败
bad_typeid	typeinfo	对某个空指针p执行typeid(*p)
bad_exception	typeinfo	当某个函数fun()因在执行过程中抛出了异常声明所不允许的异常而调用unexpected()函数时，若unexpected()函数又一次抛出了fun()的异常声明所不允许的异常，且fun()的异常声明列表中有bad_exception，则会有一个bad_exception异常在fun()的调用点被抛出
ios_base::failure	ios	用来表示C++的输入输出流执行过程中发生的错误
underflow_error	stdexcept	算术运算时向下溢出
overflow_error	stdexcept	算术运算时向上溢出
range_error	stdexcept	内部计算时发生作用域的错误
out_of_range	stdexcept	表示一个参数值不在允许的范围之内
length_error	stdexcept	尝试创建一个长度超过最大允许值的对象
invalid_argument	stdexcept	表示向函数传入无效参数
domain_error	stdexcept	执行一段程序所需要的先决条件不满足

★标准异常类的基础

- exception：标准程序库异常类的公共基类
- logic_error表示可以在程序中被预先检测到的异常
  - 如果小心地编写程序，这类异常能够避免
- runtime_error表示难以被预先检测的异常



★例12-3 三角形面积计算

- 编写一个计算三角形面积的函数，函数的参数为三角形三边边长*a*、*b*、*c*，可以用Heron公式计算：

- 设 $p=\frac{a+b+c}{2}$ ，则三角形面积 $S=\sqrt{p(p-a)(p-b)(p-c)}$



★例12-3 三角形面积计算

```
#include <iostream>  //12_3.cpp
#include <cmath>
#include <stdexcept>
using namespace std;
//给出三角形三边长，计算三角形面积
double area(double a, double b, double c)  throw (invalid_argument) {
  //判断三角形边长是否为正
  if (a <= 0 || b <= 0 || c <= 0)
    throw invalid_argument("the side length should be positive");
  //判断三边长是否满足三角不等式
  if (a + b <= c || b + c <= a || c + a <= b)
    throw invalid_argument("the side length should fit the triangle inequation");
  //由Heron公式计算三角形面积
  double s = (a + b + c) / 2;
  return sqrt(s * (s - a) * (s - b) * (s - c));
}
```

★例12-3 三角形面积计算


```
int main( ) {
    double a, b, c;       //三角形三边长
    cout << "Please input the side lengths of a triangle: ";
    cin >> a >> b >> c;
    try {
        double s = area(a, b, c);          //尝试计算三角形面积
        cout << "Area: " << s << endl;
    } catch (exception &e) {
        cout << "Error: " << e.what() << endl;
    }
    return 0;
}
```

★例12-3 三角形面积计算

运行结果1：

Please input the side lengths of a triangle: 3 4 5

Area: 6

运行结果2：

Please input the side lengths of a triangle: 0 5 5

Error: the side length should be positive

运行结果2：

Please input the side lengths of a triangle: 1 2 4

Error: the side length should fit the triangle inequation



标准异常类的使用p12_4.cpp

```
# include <iostream>
# include <new>
# include <string>
# include <stdexcept>
using namespace std;
int main() {
    string * S;
    try  {
        S=new string("ABCD");  //可能抛出bad_alloc异常
        cout<<S->substr(5,2);  //可能抛出out_of_range异常
    }
    catch(bad_alloc & NoMemory) {
        cout<<"Exception occurred: "<<NoMemory.what()<<endl;
    }
```

```
catch(out_of_range& OutOfRange)
    {
        cout<<"Exception occurred: "<<OutOfRange.what()<<endl;
    }
  return 0;
}
```

运行结果: Exception occurred: invalid string position

☞程序解释:

- **程序在使用new建立string类对象时可能因为内存不够引发异常，但程序中建立一个对象长度很小时，引发异常的可能性少。**
- **在对string类对象S取子串时，因为范围出界，5超出了字串 S 的长度，引发out_of_range类的异常。具体异常名通过调用抛出异常对象的what()函数获得。**
- **第17、21行在使用catch()捕获异常时，异常类型为引用，以便使该引用指向抛出的异常对象；而不是复制抛出的异常对象。**

12.5 本章小结

- 常见的异常有数组下标越界、运算溢出、除0、new无法获得内存、文件打开与读取不成功。异常处理的实质是使程序捕获处理错误，以不导致程序因错误中止。
- C++提供try-throw-catch的异常处理机制。
- 在函数中抛出异常使程序保持良好的结构，嵌套调用的函数内层抛出的异常从内层逐层向外传递。传递的异常被捕获，函数开始退栈过程。
- C++标准库中提供了标准异常处理类，为异常处理提供了方便。

12.6 课堂练习

- P392，习题12

